

# Guía rápida de LaTeX para Lab 4

2026

---

## 1. Objetivo de esta guía

Esta guía es para que puedan:

1. Entender los conceptos básicos de LaTeX.
  2. Editar sin miedo las plantillas de **paper** y **poster** de la materia.
  3. Configurar VS Code para trabajar cómodo con `.tex`.
  4. Instalar `pdflatex` en Linux y compilar un `.tex` a PDF.
- 

## 2. ¿Qué es LaTeX y cómo se trabaja?

LaTeX es un sistema de composición de documentos. En vez de “dibujar” el documento como en un procesador de texto, escriben texto con comandos y luego lo **compilan** para generar el PDF.

Flujo típico:

1. Editan `archivo.tex`.
  2. Ejecutan compilación (`pdflatex archivo.tex`).
  3. Obtienen `archivo.pdf`.
  4. Corrigen errores o ajustan formato y repiten.
- 

## 3. Estructura mínima de un archivo `.tex`

Ejemplo mínimo:

```
None
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[utf8]{inputenc}
```

```

\title{Ejemplo}
\author{Nombre Apellido}
\date{\today}

\begin{document}
\maketitle

Hola, este es mi primer documento en LaTeX.

\section{Introducción}
Texto de ejemplo.

\end{document}

```

Conceptos clave:

- `\documentclass{...}` define el tipo de documento.
- `\usepackage{...}` carga paquetes extra (idioma, imágenes, tablas, etc.).
- Todo el contenido va entre `\begin{document}` y `\end{document}`.
- `\section`, `\subsection`, etc. estructuran el texto.

---

## 4. Comandos básicos que van a usar siempre

- **Títulos y secciones:** `\section{}`, `\subsection{}`, `\subsubsection{}`
- **Negrita y cursiva:** `\textbf{...}`, `\textit{...}`
- **Listas:**
  - `itemize` (viñetas)
  - `enumerate` (numerada)
- **Referencias simples:** `\label{}`, `\ref{}`
- **Figuras:** entorno `figure` + `\includegraphics`
- **Tablas:** entorno `table` + `tabular`

Ejemplo corto de figura:

```

None
\begin{figure}[h]
\centering
\includegraphics[width=0.7\linewidth]{figuras/topologia.png}
\caption{Topología simulada}

```

```
\label{fig:topologia}
\end{figure}
```

---

## 5. Cómo modificar las plantillas de paper y poster

Las plantillas ya les resuelven la estética general. Lo importante es **reemplazar contenido** y no romper estructura.

### Regla general (muy importante)

- Modifiquen texto dentro de secciones ya definidas.
- No borren comandos que no entienden (`\documentclass`, `\usepackage`, bloques de formato).
- Hagan cambios de a poco y compilen seguido.

### 5.1 Plantilla de paper

Suelen editar:

- Título (`\title{...}`)
- Autores (`\author{...}`)
- Resumen / abstract
- Secciones del cuerpo (introducción, metodología, resultados, conclusiones)
- Figuras y tablas con sus captions
- Bibliografía

Buenas prácticas:

- Mantener los nombres de secciones obligatorias que pide la cátedra.
- Poner etiquetas en figuras/tablas (`\label`) y referenciarlas con `\ref`.
- Evitar copiar imágenes sin revisar resolución.

### 5.2 Plantilla de poster

Suelen editar:

- Bloques principales (objetivo, método, resultados, conclusiones)
- Imágenes/gráficos
- Título grande, autores y afiliación
- Texto breve y legible (el poster se lee de pie y a distancia)

Buenas prácticas:

- Menos texto, más visual.
- Usar viñetas y frases cortas.
- Verificar contraste y tamaño de fuente.

### 5.3 Errores comunes al tocar templates

- Borrar llaves `{ }` o comandos de cierre (`\end{...}`).
  - Cambiar nombres de archivos de imágenes sin actualizar en `\includegraphics`.
  - Usar caracteres especiales sin escape en contextos delicados (`_`, `%`, `&`).
  - Editar demasiado sin compilar y perder de vista dónde se rompió.
- 

## 6. Configurar VS Code para editar LaTeX

### 6.1 Extensión recomendada

Instalar la extensión **LaTeX Workshop** desde el marketplace de VS Code.

Pasos:

1. Abrir VS Code.
2. Ir a extensiones (`Ctrl+Shift+X`).
3. Buscar `LaTeX Workshop`.
4. Instalar.

### 6.2 Qué les aporta LaTeX Workshop

- Compilación desde VS Code.
- Visor de PDF integrado.
- Navegación entre errores de compilación.
- Autocompletado de comandos y entornos.

### 6.3 Configuración mínima sugerida

Pueden agregar en `settings.json`:

```
JSON
{
  "editor.wordWrap": "on",
```

```
"latex-workshop.latex.autoBuild.run": "onSave",  
"latex-workshop.view.pdf.viewer": "tab"  
}
```

Esto compila al guardar y abre el PDF en una pestaña dentro de VS Code.

---

## 7. Instalar pdflatex en Linux

`pdflatex` viene dentro de distribuciones TeX (TeX Live).

### Ubuntu / Debian

Instalación mínima útil:

```
Shell  
sudo apt update  
sudo apt install texlive-latex-recommended texlive-latex-extra  
texlive-fonts-recommended
```

Si quieren una instalación más completa:

```
Shell  
sudo apt install texlive-full
```

### Verificar instalación

```
Shell  
pdflatex --version
```

Si devuelve versión, está instalado correctamente.

---

## 8. Compilar un `.tex` y generar PDF

Desde terminal, ubicados en la carpeta del archivo:

```
Shell
pdflatex mi_archivo.tex
```

Resultado esperado:

- Se genera **mi\_archivo.pdf**.
- También aparecen archivos auxiliares (**.aux**, **.log**, etc.).

## Compilación recomendada para referencias cruzadas

En documentos con índice, referencias o bibliografía, conviene compilar más de una vez:

```
Shell
pdflatex mi_archivo.tex
pdflatex mi_archivo.tex
```

## Si hay bibliografía con BibTeX (si aplica)

```
Shell
pdflatex mi_archivo.tex
bibtex mi_archivo
pdflatex mi_archivo.tex
pdflatex mi_archivo.tex
```

---

## 9. Lectura rápida de errores de compilación

Cuando falle **pdflatex**, mirar primero:

1. La línea del error en terminal o en el panel de LaTeX Workshop.
2. El archivo **.log** si el mensaje no es claro.

Errores típicos:

- **Missing \$ inserted**: usaron **\_** en texto normal; escribir **\\_**.
  - **Undefined control sequence**: comando mal escrito o paquete faltante.
  - **File not found**: imagen o archivo no existe en esa ruta.
-

## 10. Checklist antes de entregar paper/poster

- Compila sin errores.
  - PDF abre bien y tiene todas las imágenes.
  - Título, autores y contenido final del grupo (sin placeholders).
  - Referencias cruzadas correctas (figuras/tablas/secciones).
  - Ortografía y consistencia de formato.
- 

## 11. Mini guía de supervivencia para templates

Trabajen con Git para mantener un historial seguro y poder volver atrás sin perder trabajo.

Flujo recomendado:

1. Ver estado antes de empezar: `git status`.
2. Hacer **un cambio chico** (por ejemplo, una sección o una figura).
3. Compilar y verificar que el PDF esté bien.
4. Guardar ese avance con un commit corto y claro:
  - `git add .`
  - `git commit -m "ajusta sección de resultados en paper"`
5. Repetir el ciclo con otro cambio chico.

Si algo se rompe, identifiquen rápido qué commit introdujo el problema y vuelvan al último estado sano.

Regla de oro: **Git + cambios chicos + compilación frecuente.**